CDA 3201L Lab #9

Field Programmable Logic Devices

Welcome to CDA3201L Lab #9! The purpose of this lab is to give you a brief introduction to programmable logic devices, and in particular Field Programmable Gate Arrays (FPGAs). You'll implement a small module in Verilog, then map the resulting configuration file to an FPGA development board.

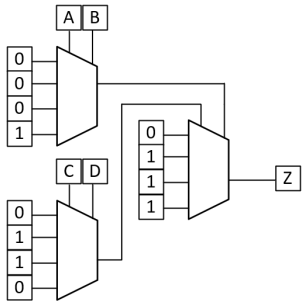
Background

FPGAs are reprogrammable logic devices that can implement any circuit (up to the size limitation of the chip) as lookup tables (LUTs). FPGAs contain Configurable Logic Blocks (CLBs) which contain these LUTs, essentially a truth table (memory) connected to a mux and a flip flop. FPGAs contain many thousands of these CLBs in a 2D grid, and the output of one CLB can be connected to the input of another to chain different logic functions together.

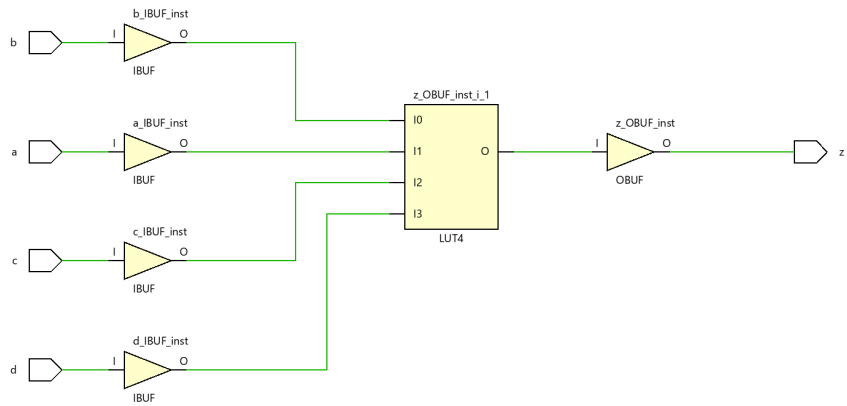
FPGA design software takes a circuit description (e.g. in Verilog or VHDL), analyzes the logic, and **synthesizes** the circuit into a representation of the original function that fits the logic constraints of the target device. This process is called **technology mapping**. Consider this example Verilog:

```
module booleanFunction1 (a, b, c, d, z);

   input a, b, c, d;
   output z;

   assign z = (a & b) | (c ^ d);

endmodule
```

The FPGA software will analyze this, optimize the logic (e.g. Quine McCluskey), and output a set of interconnected truth tables that realize the output that can fit in the logic of the target device. Most devices today have CLBs with 6-input LUTs, meaning they store every possible response for a 6-input function (so 2^6 memory elements connected to a 64-to-1 MUX). For the example here, let's imagine the target board only supports 2-input LUTs. The function above will be mapped to truth tables, where the LUTs implement A&B, C^D, and finally ORs the two outputs together:



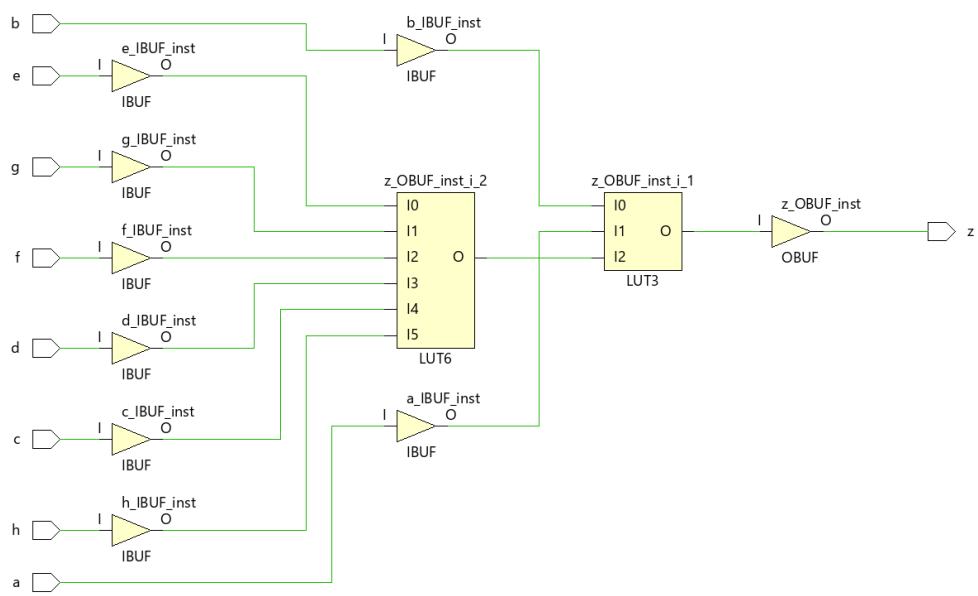
As it happens, the target board supports 6-input LUTs, so this simpler 4-input function can be mapped into a single LUT, e.g.:



Here you see the inputs A, B, C, D, the input buffers (needed for a real device, unlike our simple example earlier), the LUT (in this case, LUT4, since there are 4 inputs), output buffer, and the output Z. In an example with >6 variables, one LUT cannot hold all possible responses, so it is distributed, e.g.:

```
module lutex(a, b, c, d, e, f, g, h, z);

   input a, b, c, d, e, f, g, h;
   output z;

   assign z = (a & b) | (c ^ d) ^ (e | (f & g)) ^ h;

endmodule
```

which results in the following schematic:



Here you can see a LUT6 which implements a 6-input function for variables c, d, e, f, g, and h, the output of which is fed into a 3-input function along with a and b, to produce the output z.

Here are the actual functions for the LUT6 and LUT3. Note that I've changed the name of the outputs to O_6 and O_3 for clarity:

O_6 = ed'c'h' + gfd'c'h' + e'f'dc'h' + e'g'dc'h' + e'f'd'ch' + e'g'd'ch' + edch' + gfdch' + e'f'd'c'h + e'g'd'c'h + edc'h + gfdc'h + ed'ch + gfd'ch + e'f'dch + e'g'dch

O_3 = b & a + O_6

Here are the last few entries in the corresponding LUT6 truth table. As you can see, the last term in O_6 (e'g'dch) covers the two 1s shown in the output column below:

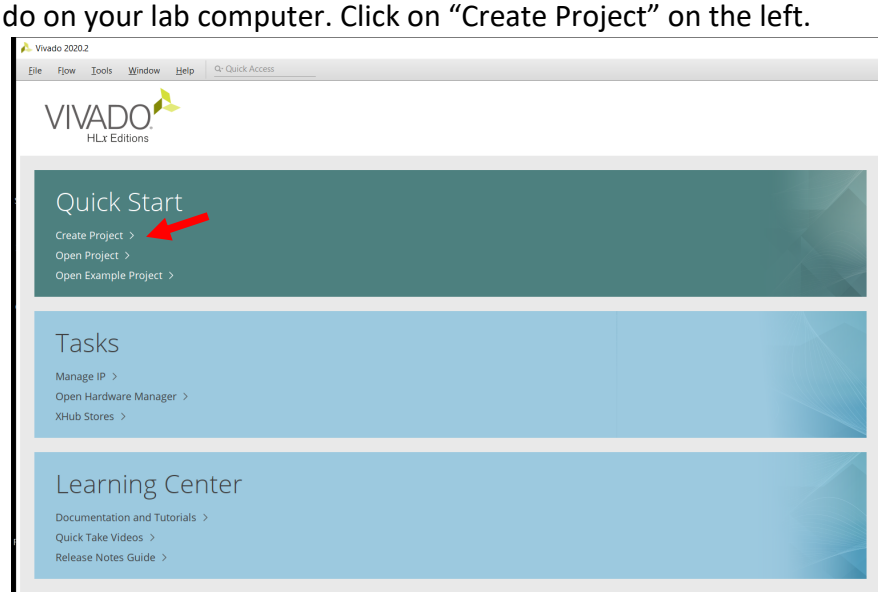
#	h	c	d	f	g	e	O
58	1	1	1	0	1	0	1
59	1	1	1	0	1	1	0
60	1	1	1	1	0	0	1
61	1	1	1	1	0	1	0
62	1	1	1	1	1	0	0

The system automatically optimized the logic function, broke it down, generated the truth tables, determined a way to connect them that met the requirements (area, power, timing), and generated a binary configuration file (**bitstream**) that implements the circuit described in the Verilog module. *Any logic function, any state machine, including something as complex as an entire processor, can be implemented this way!*

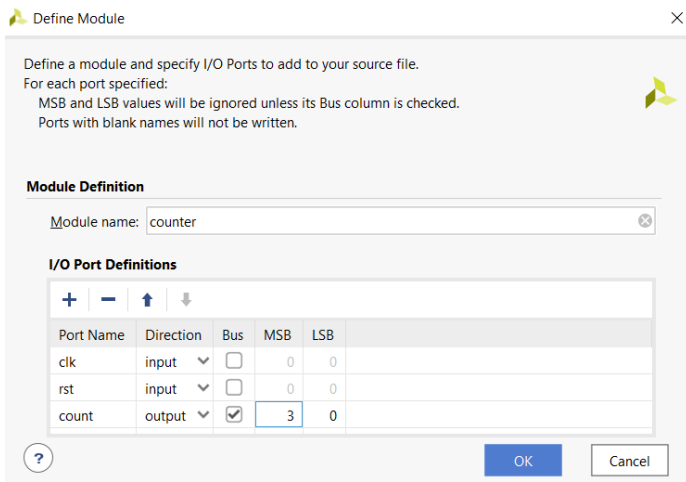
Walkthrough

The FPGA software we will use is Vivado and is made by Xilinx, the same manufacturer of the FPGA chip itself. The following instructions will guide you through the process of creating a project, adding the necessary files, and programming your board.

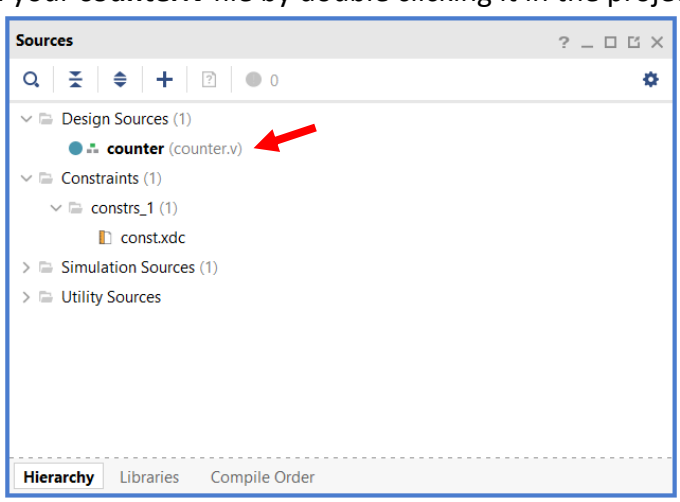
1. Open Vivado on your lab computer. Click on "Create Project" on the left.



2. This opens the New Project Wizard. Set the project name and location. Leave the "create project subdirectory" box checked. Then, proceed through the next project settings prompts, hitting "next" after you are done with each step:
    a. Set the Project Type to "RTL Project". Do not check other boxes.
    b. Click "Create File". Set the file type (Verilog). Name the file **counter.v** and click OK.
    c. Click "Create File" to create a constraints (XDC) file. You can name this **const** (for constraints). Click OK.
    d. You now have the option to choose a part. There are A LOT of different supported FPGA devices. For this one we want the xc7s25csga225-1. You can start typing the name and it will filter the list. Make sure you choose the correct one! Notice the chip properties, # pins, # I/Os, # LUTs, FFs, etc. Note these down.
    e. Continue to the confirmation screen and click Finish. Vivado will now set up your project.
3. You'll have the option to set up the top level module in your file **counter.v**. You can do so now. Include clk (input), rst (input), and count (output, bus, 3:0). Click OK.



4. You can now open your **counter.v** file by double clicking it in the project explorer:

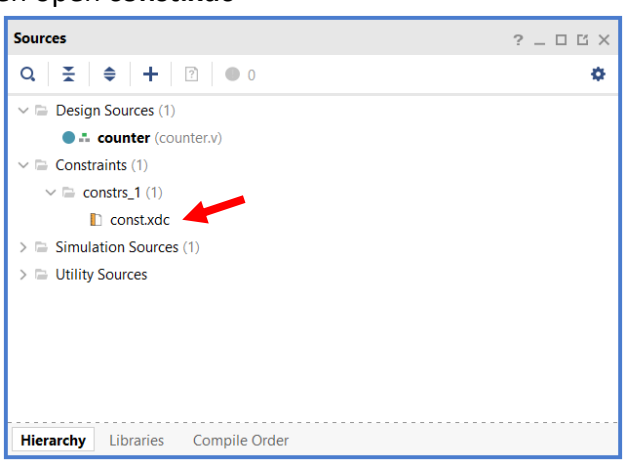


5. During the Module Definition (Step 3), we defined an output “count”, but there was no option to set it as a reg. Make this change manually in the Verilog. Then, enter the following code (a basic counter, similar to what you have previously implemented):

```
`timescale 1ns / 1ps

module counter(
    input clk,
    input rst,
    output reg [3:0] count
    );

    reg [22:0] clk_div;
    wire slow_clk;

    assign slow_clk = clk_div[22];

    always @ (posedge clk) begin
        if (rst == 1'b1) begin
            clk_div = 0;
        end
        else begin
            clk_div = clk_div + 1;
        end
    end

    always @ (posedge slow_clk or posedge rst) begin
        if (rst == 1'b1) begin
            count = 0;
        end
        else begin
            count = count + 1;
        end
    end

endmodule
```

There are two things going on here. At the first arrow, we have a procedure implementing a 23-bit counter with a synchronous active high reset that increments on the rising edge of the **clk** signal. The **clk** signal comes from the board’s clock pin, which runs at 12 MHz. At 12 MHz, we get a new positive edge every ~83.3 ns. We need to slow this down considerably because we want to output the **count** register to the on-board LEDs. If we divide this by 2, we get a 6 MHz clock. If we divide it by 2 again (21 more times), we get a clock of around 2 Hz which is much more manageable (we can see it).

6. Save the file, and then open **const.xdc**



7. The constraints file is extremely important – without it, the software would not know how to generate the configuration file that would work with the specific device. FPGAs

are physical devices; the constraints file, among other things, lets you define which physical pin is mapped to the “pins” in the module port list. In other words, where should the device send the output of count[0]? Where does it get the input for “rst”? The following constraints can be used for this project:

```
## 12 MHz System Clock
set_property -dict { PACKAGE_PIN M9 IOSTANDARD LVCMOS33 } [ get_ports { clk }];
create_clock -add -name sys_clk_pin -period 83.33 -waveform {0 41.66} [ get_ports { clk }];

## Push Button for reset
set_property -dict { PACKAGE_PIN D2 IOSTANDARD LVCMOS33 } [ get_ports { rst }];

## 4 LEDs
set_property -dict { PACKAGE_PIN E2 IOSTANDARD LVCMOS33 } [ get_ports { count[0] }];
set_property -dict { PACKAGE_PIN K1 IOSTANDARD LVCMOS33 } [ get_ports { count[1] }];
set_property -dict { PACKAGE_PIN J1 IOSTANDARD LVCMOS33 } [ get_ports { count[2] }];
set_property -dict { PACKAGE_PIN E1 IOSTANDARD LVCMOS33 } [ get_ports { count[3] }];
```

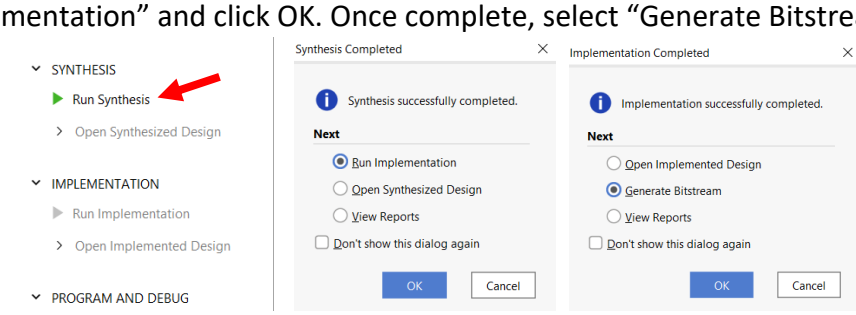
The first line sets the pin for the clock. It specifies that the Verilog port “clk” should be connected with the physical pin M9 on the FPGA. On the board, pin M9 is connected to the 12 MHz oscillator, which is the clock source for the chip.

The next line defines the properties of the clock. The software has no idea the board has a 12 MHz clock – you must tell it this. Changing this does NOT change the clock frequency, because this information is used simply to check the timing of the design (i.e. it checks for setup/hold violations for the flip flops).

The next lines define the pin for the pushbutton (there are 2 on the board). One will be the source for the reset signal. Finally, the next four lines connect the LED pins (E2, K1, J1, and E1) to specific bits of the **count** register.

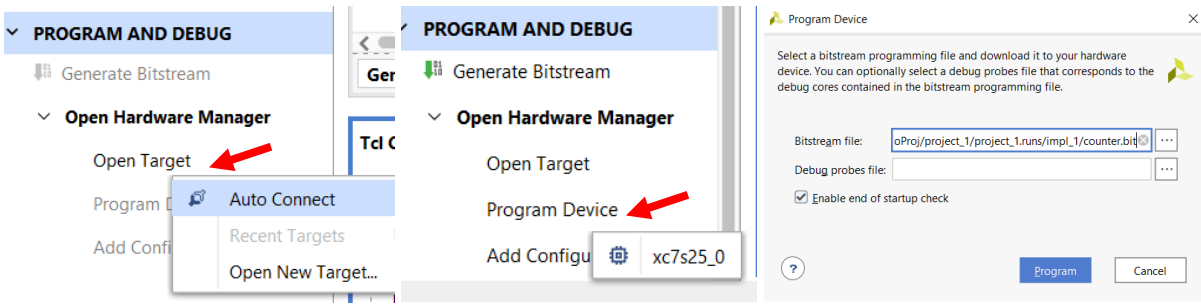
<u>Copy and paste these into the const file and save.</u>

8. Now, if everything has been entered correctly up to this point, you should be able to start the synthesis process. Click on “Run Synthesis” on the left pane. If that is successful, you should get a window asking you what to do next. Leave it on “Run Implementation” and click OK. Once complete, select “Generate Bitstream” / click OK.



9. The bitstream is the binary configuration file that defines what bits are in what LUTs, what LUTs connect to what other LUTs (and how), what is stored in the memories, and so on. It contains the entire design in a format that is only understood by this specific FPGA (the one you selected in Step 2d). You can program the device with this bitstream.

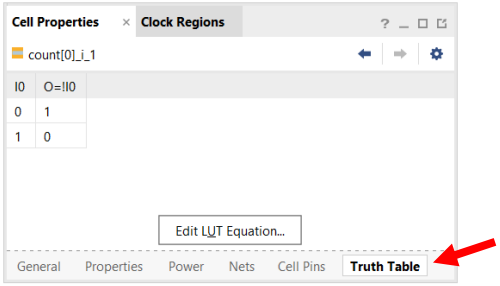
Connect the FPGA to the computer using the USB cable. You should see some status lights showing it has power. Click on “Open Hardware Manager” then “Open Target” and select “Auto Connect”. After a few seconds it should find the device and give you the option to program the device. Clicking on “Program Device” should show the board xc7s25 (the board family) and _0, showing it is the only connected device. Clicking this will bring up the Program Device window; **counter.bit** is already selected, so press OK.



10. Now that the device is programmed, you should immediately see the LEDs incrementing in binary. Pressing the pushbutton farthest from the USB port should reset the counter. As long as the button is held, it should not count (rst == 1). Once you release, it will begin incrementing, which you can see on the LEDs.

<u>Lab Assignment/Questions</u>

1. (5 pts) In your own words, explain how an FPGA works. What kind of circuit can it implement?
2. (5 pts) In the Verilog module, what kind of reset (active high, active low, synchronous, asynchronous) was used? What practical effect did this have on the functionality of the circuit / user interface?
3. (5 pts) In Vivado, on the left side, click on “Open Implemented Design” and then on “Schematic”. Take a screenshot of the schematic for the report.
4. (5 pts) Zoom in on the schematic. Find the LUT labeled “count[0]_i_1”. On the left side, there is a window called “Cell Properties”, shown here. Click on “Truth Table”



This displays the truth table implemented inside that LUT. Here, for example, it is a 1 input LUT that implements an inverter. Find the LUTs “count[1]_i_1” and “count[2]_i_1” in the schematic. Clicking on one will show its truth table in the Cell Properties window. What functions do they implement? Does this make sense for a counter? Why/why not?

Attention: The deliverable for this lab is your answers for Q1, Q2, Q3, and Q4 in the Lab Assignment/Questions section. No formal report will be needed at this time. However, the instructions below must be followed when preparing your derivable:

1. Answers must be given in a complete manner, and work must be always shown. Answers with no work shown will be given a grade of *zero* points;
2. The presentation will be taken into consideration when grading (i.e., points may be deducted from unreadable/dubious handwriting, illegible drawings, etc.);
3. No late submissions will be allowed (please see the Syllabus on Canvas for more information on the late submission policy and possible excuses).

Your deliverable (i.e., answers for Q1, Q2, Q3, and Q4) must be submitted through Canvas.